

Advanced Programming

Software Design Patterns
Architectural Patterns

Álvaro Santos@DEIS-ISEC

Software Design Patterns

Factory method

Factory method pattern

- The *Factory* pattern, also known as the *Factory method*, provides a mechanism (method) that allows the creation of an instance of a class from a given set of possibilities
 - It's typically applied in a context where there's a hierarchy, from which a set of subclasses derive, and this pattern is used as a mechanism to create an instance of one of these subclasses

Factory method pattern

- Usual implementation:
 - A `static` method is defined
 - The method can be created in several places (choosing the most adequate place for each project). Examples:
 - In the interface or base class of the hierarchy where the types of objects to be created are derived from
 - In a class created for this purpose (for example: *PieceFactory*)
 - In a Java *enum*
 - The method receives as parameters:
 - The necessary data to determine the desired object type
 - The parameters required for its instantiation
 - The method returns an instance of the base type that represents all possible types created by the factory

Example (base class/interface)

```
enum PieceType {  
    KING, QUEEN, BISHOP, KNIGHT, ROOK, PAWN  
}  
  
public abstract class Piece {  
    ...  
    static Piece createPiece( PieceType type, Board board,  
                           Player player, int x, int y) {  
        return switch (type) {  
            case KING   -> new King(board,player,x,y) ;  
            case QUEEN  -> new Queen(board,player,x,y) ;  
            ...  
            // default -> null;  
        };  
    }  
}
```

Example (independent class)

```
enum PieceType {  
    KING, QUEEN, BISHOP, KNIGHT, ROOK, PAWN  
}  
  
class PieceFactory {  
    static Piece createPiece( PieceType type, Board board,  
                             Player player, int x, int y) {  
        return switch (type) {  
            case KING   -> new King(board,player,x,y) ;  
            case QUEEN  -> new Queen(board,player,x,y) ;  
            ...  
            // default -> null;  
        };  
    }  
}
```

Example (enum – static)

```
enum PieceType {  
    KING, QUEEN, BISHOP, KNIGHT, ROOK, PAWN;  
  
    //Factory: static method  
    static Piece createPiece( PieceType type, Board board,  
                              Player player, int x, int y) {  
        return switch (type) {  
            case KING   -> new King(board,player,x,y);  
            case QUEEN  -> new Queen(board,player,x,y);  
            ...  
            // default -> null;  
        };  
    }  
}
```

Example (enum – instance)

```
enum PieceType {  
    KING, QUEEN, BISHOP, KNIGHT, ROOK, PAWN;  
  
    //Factory: instance method  
    Piece createPiece(Board board, Player player, int x, int y) {  
        return switch (this) {  
            case KING    -> new King(board,player,x,y);  
            case QUEEN   -> new Queen(board,player,x,y);  
            ...  
            // default -> null;  
        };  
    }  
}
```

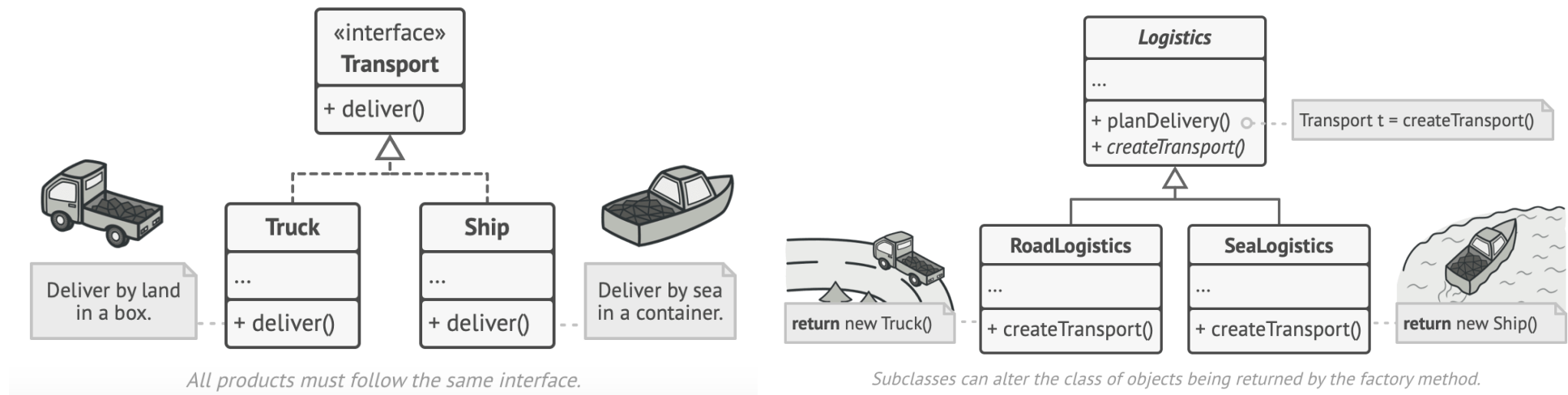
Example (enum – instance v2)

```
public enum PieceType {
    KING {
        @Override
        Piece createPiece(Board board, Player player, int x, int y) {
            return new King(board, player, x, y);
        }
    },
    QUEEN {
        @Override
        Piece createPiece(Board board, Player player, int x, int y) {
            return new Queen(board, player, x, y);
        }
    },
    ... ;

    abstract Piece createPiece(Board board, Player player, int x, int y);
}
```

Polymorphic Factory

- The pattern can be implemented in other ways, for example, by using instance methods in subclasses to create the object instances
 - This form allows the redefinition of these methods in the derived classes (to include other types of objects in the factory)



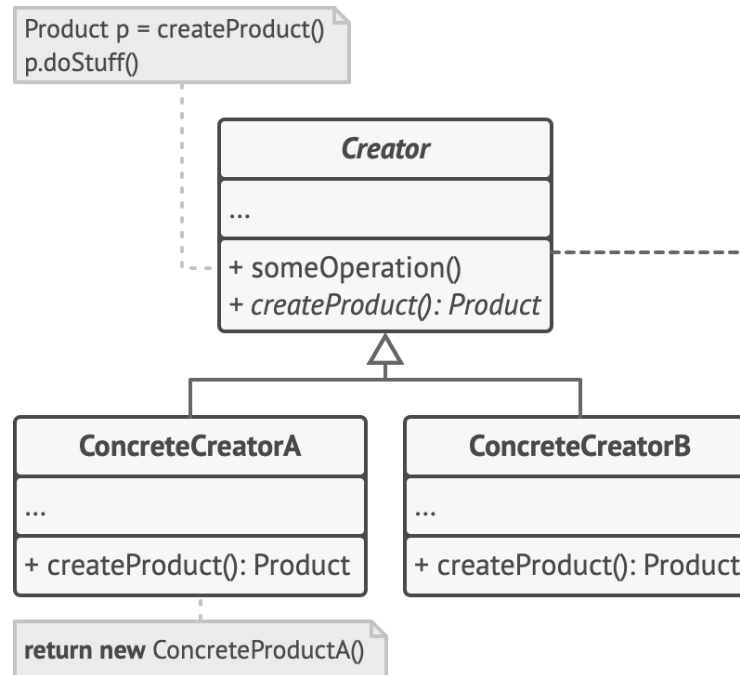
<https://refactoring.guru/design-patterns/factory-method>

Polymorphic Factory

3 The **Creator** class declares the factory method that returns new product objects. It's important that the return type of this method matches the product interface.

You can declare the factory method as `abstract` to force all subclasses to implement their own versions of the method. As an alternative, the base factory method can return some default product type.

Note, despite its name, product creation is **not** the primary responsibility of the creator. Usually, the creator class already has some core business logic related to products. The factory method helps to decouple this logic from the concrete product classes. Here is an analogy: a large software development company can have a training department for programmers. However, the primary function of the company as a whole is still writing code, not producing programmers.



1 The **Product** declares the interface, which is common to all objects that can be produced by the creator and its subclasses.

2 **Concrete Products** are different implementations of the product interface.

4 **Concrete Creators** override the base factory method so it returns a different type of product.

Note that the factory method doesn't have to **create** new instances all the time. It can also return existing objects from a cache, an object pool, or another source.

<https://refactoring.guru/design-patterns/factory-method>

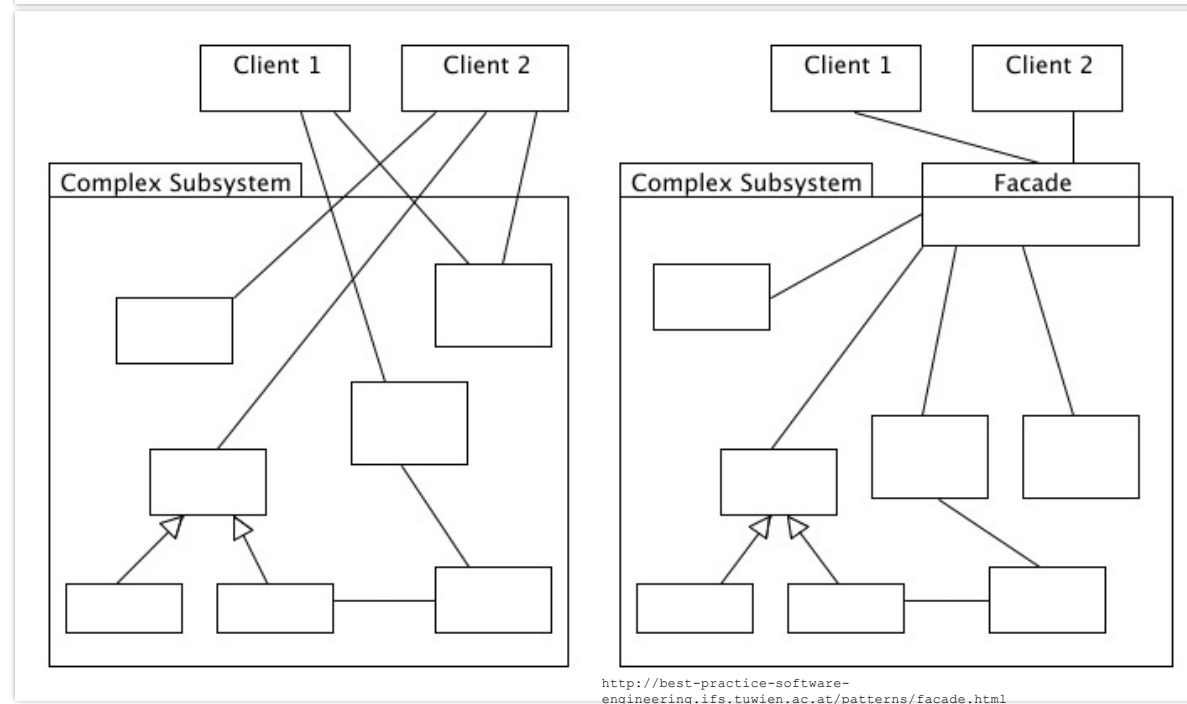
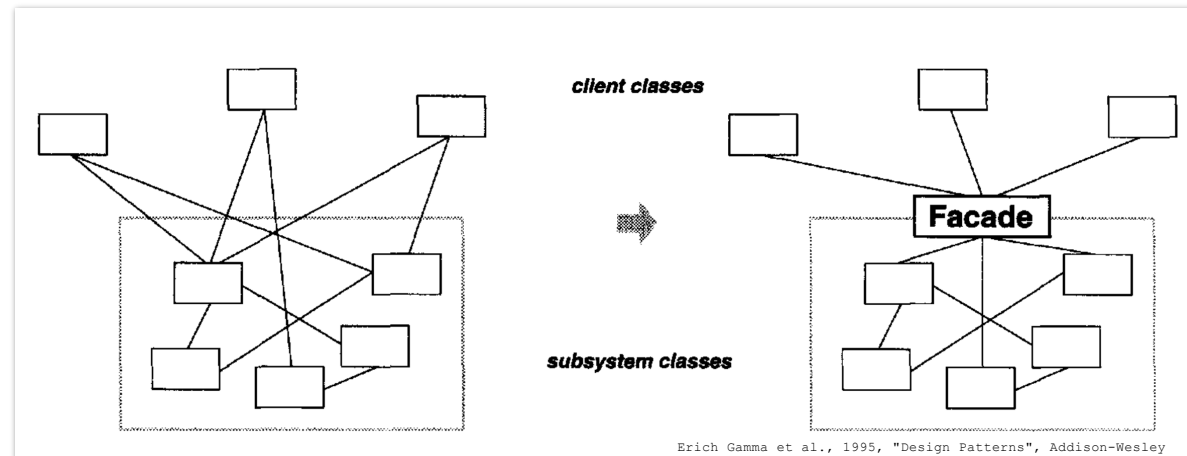
Software Design Patterns

Facade pattern

Facade

- The use of complex data structures and algorithms that rely on multiple classes within an application's data model can – and should – be hidden from the classes that utilize their functionalities, such as user interface classes
- Implementation details can be hidden by introducing an intermediary class that serves as an access point to the functionalities provided by these systems, classes, or algorithms
- This simplified access interface is called a *Facade*

Facade



Facade

- A *Facade* class provides a simplified interface to a complex subsystem by exposing higher-level methods that internally delegate calls to the underlying components
- Its primary role is to limit direct access to the internal objects of a *package* while maintaining controlled visibility
- Although its main goal is to simplify access, a *Facade* can also introduce new functionalities, such as combining multiple operations, enforcing constraints, or providing default configurations